

DSA Project – Algorithm Analysis & Trace Documentation

Individual Documentation – Dzifa

This report documents the shortest-path tracing, MST algorithm comparison, correctness testing, and performance experiment carried out for the implemented Dijkstra, Prim, and Kruskal algorithms.

1. Dijkstra Graph Used for Tracing

The connected test graph used in the implementation contains the weighted edges 0–1 (4), 0–2 (1), 1–2 (2), 1–3 (1), and 2–3 (5). The source vertex is 0.

2. Trace Table 1 – getPath(1)

Step	Current vertex	previous[] effect	Path reconstruction
1	1	previous[1] = 2	1 → 2
2	2	previous[2] = 0	2 → 0
3	0	previous[0] = -1	0 reached

Result: getPath(1) returns [0, 2, 1], giving the route 0 → 2 → 1. The shortest distance to vertex 1 is 3.

3. Trace Table 2 – getPath(3)

Step	Current vertex	previous[] effect	Path reconstruction
1	3	previous[3] = 1	3 → 1
2	1	previous[1] = 2	1 → 2
3	2	previous[2] = 0	2 → 0
4	0	previous[0] = -1	0 reached

Result: getPath(3) returns [0, 2, 1, 3], giving the route 0 → 2 → 1 → 3. The shortest distance to vertex 3 is 4.

4. Dijkstra Final Distance Results

Destination	Distance	getPath() result
0	0	[0]
1	3	[0, 2, 1]
2	1	[0, 2]
3	4	[0, 2, 1, 3]

5. Prim vs Kruskal Comparison

Both algorithms were applied to the same five-edge weighted graph. They produced minimum spanning trees with three edges and the same total MST weight of 4, although the order of the selected edges differs.

Algorithm	Selected edges	Total weight
Prim	0–2 (1), 2–1 (2), 1–3 (1)	4

Kruskal	0–2 (1), 1–3 (1), 1–2 (2)	4
---------	---------------------------	---

Theoretical complexity: Prim is documented as $O(E \log V)$, while Kruskal is documented as $O(E \log E)$. Prim grows from a connected vertex using the cheapest available boundary edge, whereas Kruskal considers edges in increasing weight order and uses a disjoint-set structure to avoid cycles.

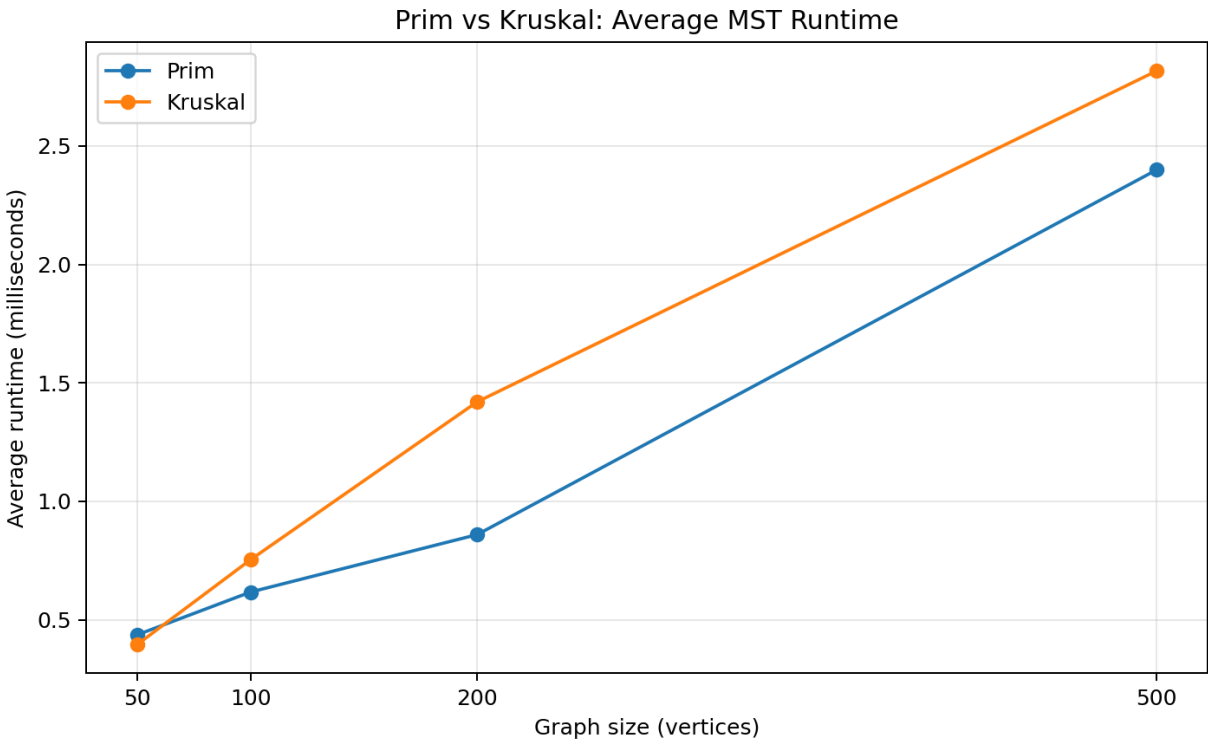
6. Correctness Testing

Algorithm	Tests	Result
Dijkstra	Normal; disconnected vertex; invalid input	3/3 PASS
Prim	Normal; single vertex; disconnected graph	3/3 PASS
Kruskal	Normal; disconnected graph; empty graph	3/3 PASS
Overall	All algorithm tests	9/9 PASS

7. Performance Experiment

The benchmark compared Prim and Kruskal on graph sizes 50, 100, 200, and 500. Each size was measured three times. Runtime was recorded using Java's `System.nanoTime()` and reported below in milliseconds as the average of the three runs.

Size	Prim avg (ms)	Kruskal avg (ms)
50	0.4364	0.3957
100	0.6170	0.7536
200	0.8593	1.4199
500	2.3995	2.8172



8. Interpretation

The measured runtimes generally increase as the graph size increases, which is consistent with both algorithms becoming more computationally demanding on larger inputs. The measurements are not perfectly

monotonic because these are small JVM benchmarks and are affected by factors such as JIT compilation, JVM scheduling and machine background activity. The results therefore illustrate observed performance rather than proving asymptotic complexity. At 500 vertices, Prim averaged 2.3995 ms, while Kruskal averaged 2.8172 ms.

9. Individual Contribution – Dzifa

My contribution focused on the graph algorithms assigned to me: Dijkstra, Prim and Kruskal. I implemented/verified the algorithms and their interaction with the project's Graph, Heap and DisjointSet data structures, added algorithm test cases, verified shortest-path reconstruction through `getPath()`, checked normal and edge cases, and performed the Prim-versus-Kruskal performance comparison. I also prepared the trace tables and supporting analysis for this documentation.

10. Conclusion

The implemented algorithms produced the expected results on the project test graph. Dijkstra correctly reconstructed the shortest paths, while Prim and Kruskal both returned an MST of total weight 4. All nine algorithm tests passed. The performance experiment showed increasing runtime with larger graph sizes, while also demonstrating that small timing experiments can fluctuate due to runtime and system effects.